



UNIVERSITÀ DI TRENTO

Relazione progetto Reti Neurali

Programmazione avanzata ed intelligenza artificiale

Studente: Daniele Pizzoli

Matricola: 243473

Corso: Programmazione avanzata ed
intelligenza artificiale
Docente: Prof. Paolo Rech

29/05/2026

Contents

1	Introduzione	3
2	Perché una CNN per l'audio	3
2.1	Spettrogramma di MEL	3
3	Dataset e preprocessing	4
4	Architetture considerate	4
5	Andamento dell'addestramento	5
6	Analisi delle prestazioni	7
6.1	Analisi degli errori	9
6.2	Considerazioni sulla scelta del modello	9
7	Conclusioni	10

1 Introduzione

Il progetto scelto è la classificazione di comandi vocali, quindi un problema di classificazione multiclasse. Il dataset adottato è Google SpeechCommands v0.02 e le classi considerate sono dieci: `yes`, `no`, `up`, `down`, `left`, `right`, `on`, `off`, `stop` e `go`.

2 Perché una CNN per l'audio

La scelta di una CNN è naturale perché, una volta trasformato il segnale in spettrogramma Mel, il problema assume molte caratteristiche di una classificazione su immagini. L'audio non viene più interpretato come una sequenza grezza di campioni, ma come una rappresentazione tempo-frequenza ottenuta con la STFT (Short-Time Fourier Transform) 1 e compressa nello spazio di Mel. Il risultato è una matrice bidimensionale in cui l'asse orizzontale rappresenta il tempo e l'asse verticale la distribuzione dell'energia sulle frequenze.

$$STFT\{x(t)\}\{\tau, \omega\} := \int_{-\infty}^{\infty} x(t)w(t - \tau)e^{-j2\pi\omega t}dt \quad (1)$$

Questa trasformazione è il punto chiave: la CNN può sfruttare filtri locali per individuare pattern ripetuti o significativi, senza dover modellare esplicitamente ogni istante della sequenza. In pratica, il problema si avvicina molto alla visione artificiale, perché il modello può cercare bordi, bande, transizioni e strutture armoniche che caratterizzano la pronuncia di una parola.

Un altro vantaggio importante è la condivisione dei pesi: lo stesso kernel viene applicato su tutta la rappresentazione, quindi la rete riesce a cercare gli stessi indizi acustici in posizioni diverse dello spettrogramma. Questo è utile sia rispetto al tempo, perché la stessa parola può comparire all'inizio o alla fine del clip, sia rispetto alla frequenza, perché una stessa pronuncia può apparire leggermente più in alto o più in basso in funzione della voce o delle condizioni di registrazione. Il pooling riduce la sensibilità a piccole traslazioni e rende il modello più robusto rispetto a variazioni di pronuncia, velocità e allineamento temporale.

2.1 Spettrogramma di MEL

Uno spettrogramma di Mel (dalla parola melody) è una rappresentazione visiva del suono che mappa le frequenze sull'asse verticale usando la scala Mel, 2 la quale simula il modo in cui l'orecchio umano percepisce l'altezza (più dettagliata sui bassi, meno sulle alte frequenze).

$$mel(f) = \begin{cases} f & \text{se } f \leq 1\text{kHz} \\ 2595 \cdot \log\left(1 + \frac{f}{700}\right) & \text{se } f > 1\text{kHz} \end{cases} \quad (2)$$

In questo modo, la CNN può concentrarsi su pattern acustici più utili per distinguere i comandi vocali, migliorando la capacità di generalizzazione e riducendo il rumore.

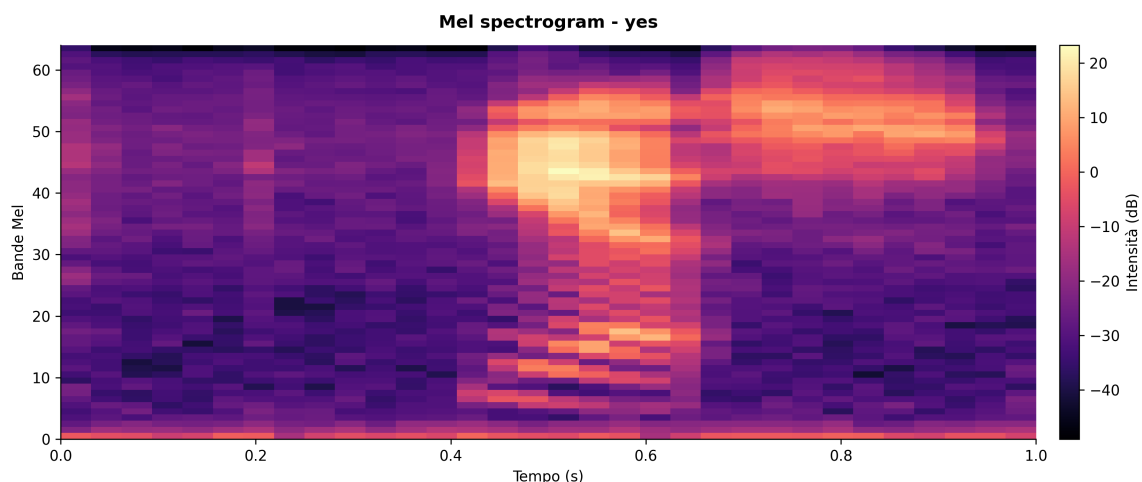


Figure 1: Esempio di spettrogramma di Mel ottenuto da un comando vocale del dataset SpeechCommands.

3 Dataset e preprocessing

Il dataset viene filtrato rispetto a quello originale per tenere solo dieci classi e alleggerire il training.

Ogni segnale audio viene portato a una frequenza di campionamento di 16 kHz e normalizzato a una durata fissa di un secondo. Questa scelta semplifica il training e rende l'input compatibile con il modello, che si aspetta tensori di dimensione costante. La conversione in spettrogramma Mel è gestita da `audio_utils.py`, dove l'audio viene trasformato con `MelSpectrogram` e convertito in dB tramite `AmplitudeToDB`.

In pratica, ogni clip viene prima adattata alla stessa lunghezza temporale e poi rappresentata come immagine monocanale, così da rendere applicabili architetture convoluzionali nate per la visione artificiale.

4 Architetture considerate

Le architetture usate nel progetto sono definite in `models.py`. In breve:

- **LightCNN**: rete base e molto semplice, con due blocchi convoluzionali e un classificatore fully connected. È il nostro modello di riferimento: veloce e leggero.

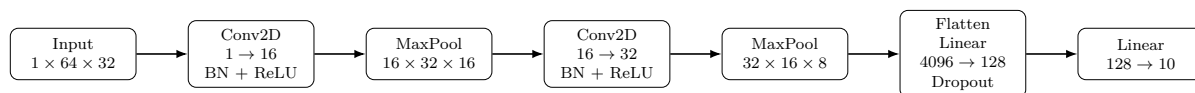


Figure 2: Architettura del modello LightCNN

- **AudioResNet** e **AudioMobileNetV2**: consigliate nelle slide del corso; sono più complesse di LightCNN e offrono migliori capacità rappresentative a costo di maggiore complessità computazionale.

- **AudioAlexNet**: suggerita da un collega di magistrale come potenzialmente migliore.

Tutte le architetture condividono l'obiettivo di estrarre feature locali dagli spettrogrammi e trasformarle in rappresentazioni utili per la classificazione, con diversi compromessi tra semplicità, tempo di addestramento e accuratezza.

5 Andamento dell'addestramento

Il confronto tra i quattro log mostra che tutti i modelli convergono in modo regolare, ma con prestazioni differenti: **AudioResNet** raggiunge il miglior risultato finale con una validation accuracy del 94.17%, seguito da **AudioMobileNetV2** con 91.71%, **LightCNN** con 90.98% e **AudioAlexNet** con 89.36% come miglior valore osservato.

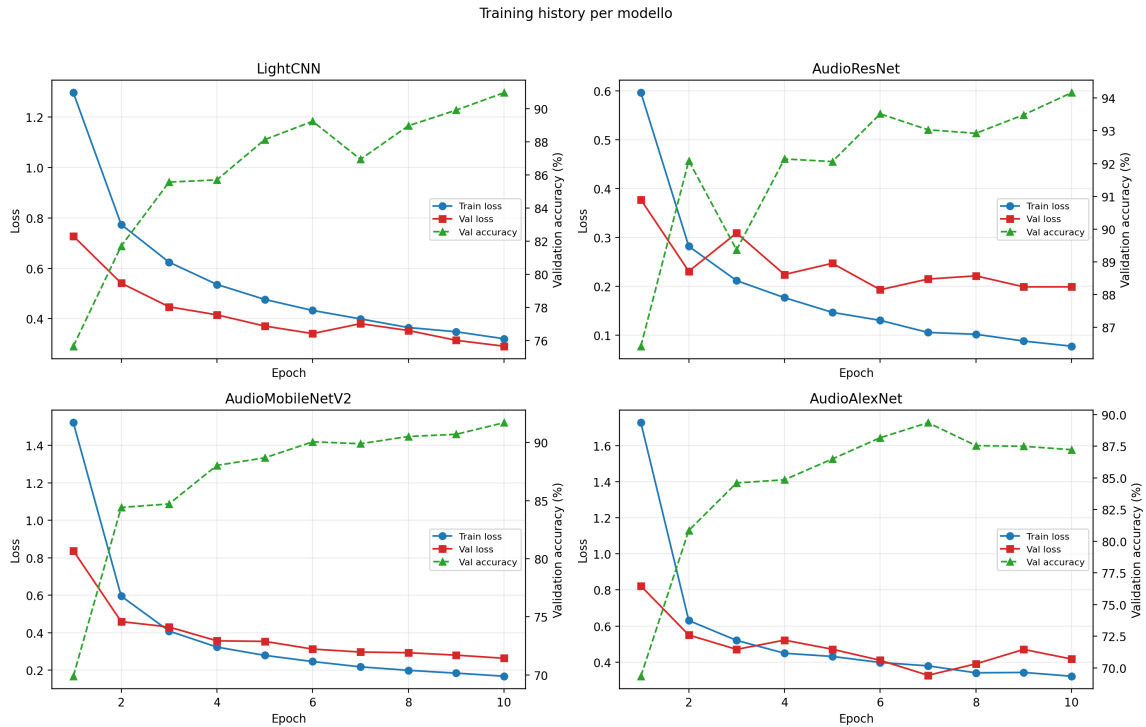


Figure 3: Andamento di training loss, validation loss e validation accuracy per ciascun modello.

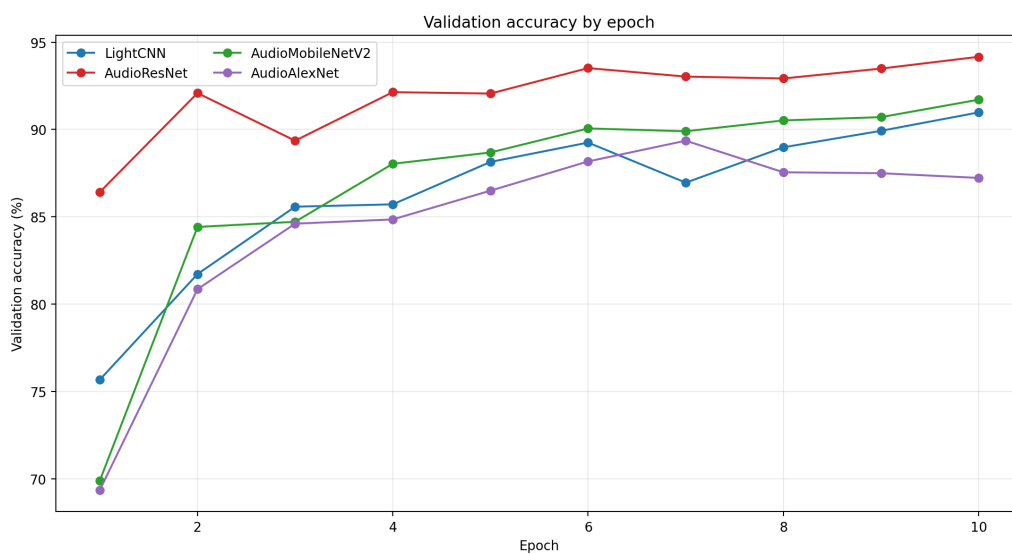


Figure 4: Confronto della validation accuracy tra i modelli addestrati.

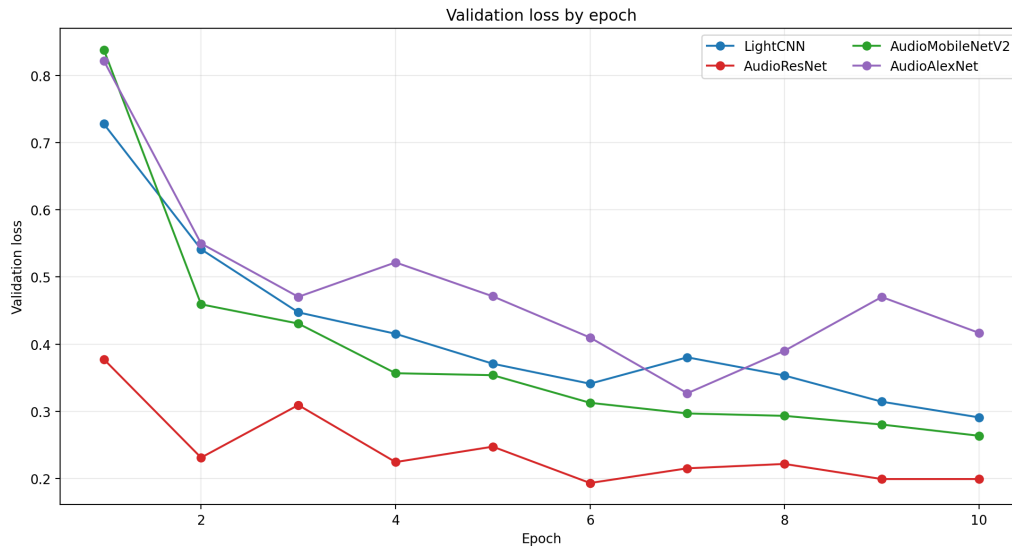


Figure 5: Confronto della validation loss tra i modelli addestrati.

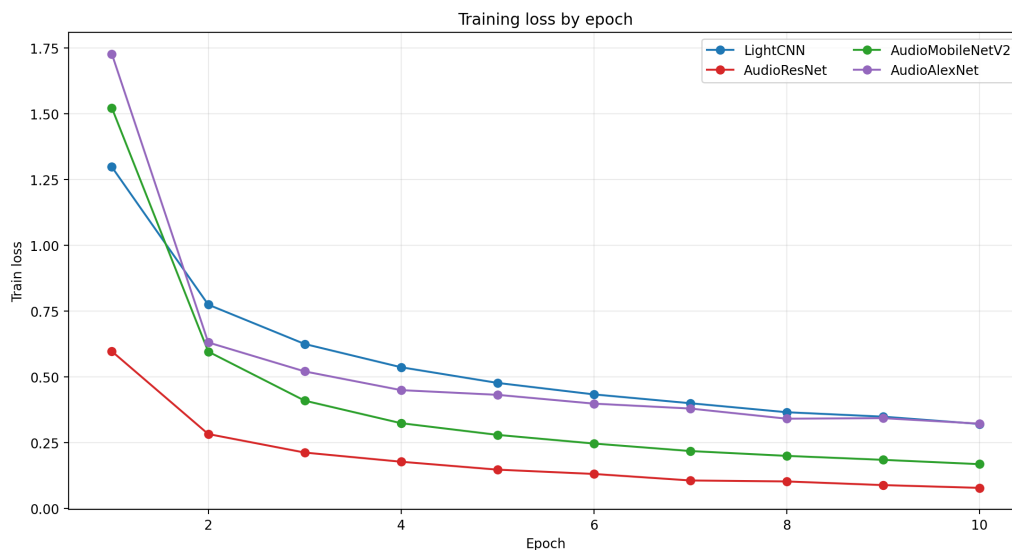


Figure 6: Confronto della training loss tra i modelli addestrati.

La Figura 3 permette di vedere insieme come gli errori diminuiscono e la precisione migliora durante l'addestramento dei vari modelli.

La Figura 4 mostra in modo chiaro quale modello funziona meglio. Si vede che ResNet è il più preciso. I grafici mostrano anche che i modelli migliori non solo sono più precisi, ma hanno un comportamento più stabile e regolare. AlexNet invece ha un comportamento più disordinato e smette di migliorare prima degli altri e risulta con performance comunque valide ma inferiori rispetto agli altri, probabilmente il modello non è adatto ai dati audio.

6 Analisi delle prestazioni

Matrice di confusione di LightCNN

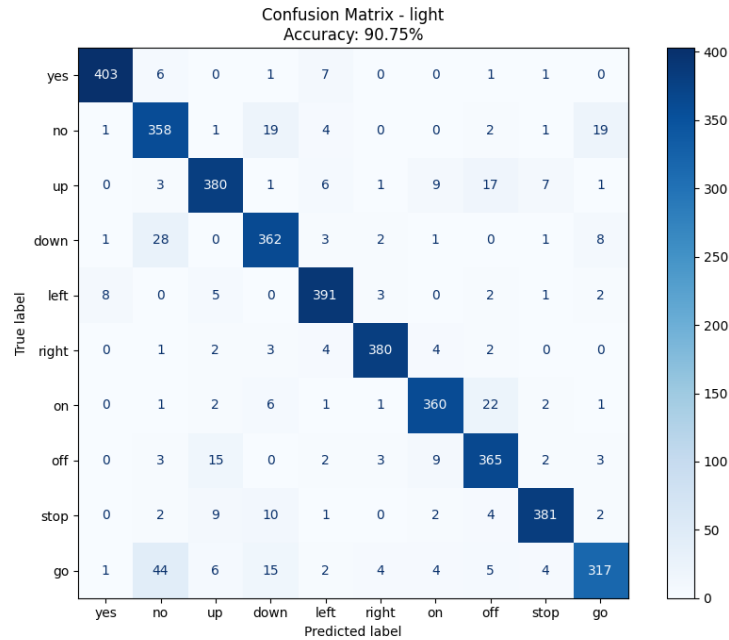


Figure 7: Matrice di confusione del modello LightCNN.

La matrice di confusione mostrata in Figura 7 evidenzia che il modello leggero riesce a riconoscere correttamente una parte consistente dei comandi, ma mostra ancora errori concentrati tra classi vicine dal punto di vista acustico. In particolare, i comandi brevi e molto simili tra loro tendono a essere scambiati con maggiore frequenza, segnale che la rete cattura bene le strutture più evidenti ma fatica nei casi ambigui.

Matrice di confusione di AudioResNet

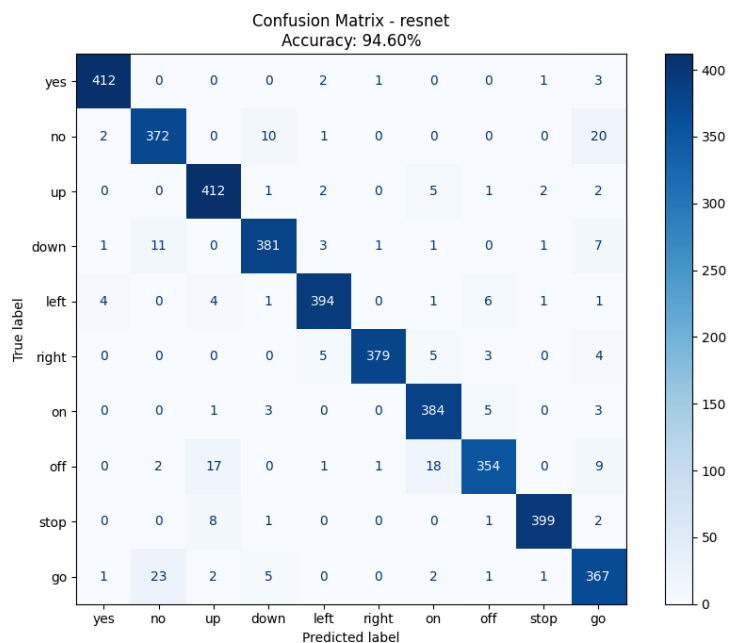


Figure 8: Matrice di confusione del modello AudioResNet.

Rispetto al modello leggero, la ResNet offre una rappresentazione più ricca e in genere produce una separazione più netta tra le classi. La Figura 8 mostra comunque che anche una rete più profonda non elimina del tutto le confusioni tra comandi simili.

Matrice di confusione di AudioMobileNetV2

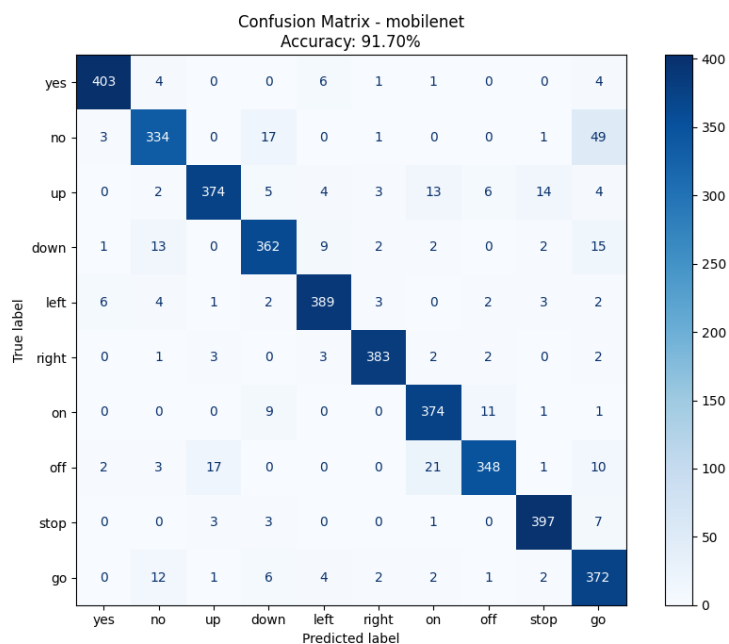


Figure 9: Matrice di confusione del modello AudioMobileNetV2.

AudioMobileNetV2 mostra un comportamento intermedio tra LightCNN e ResNet: il modello mantiene una buona capacità di discriminazione, ma alcune classi vicine restano ancora difficili da separare.

Matrici di confusione di AudioAlexNet

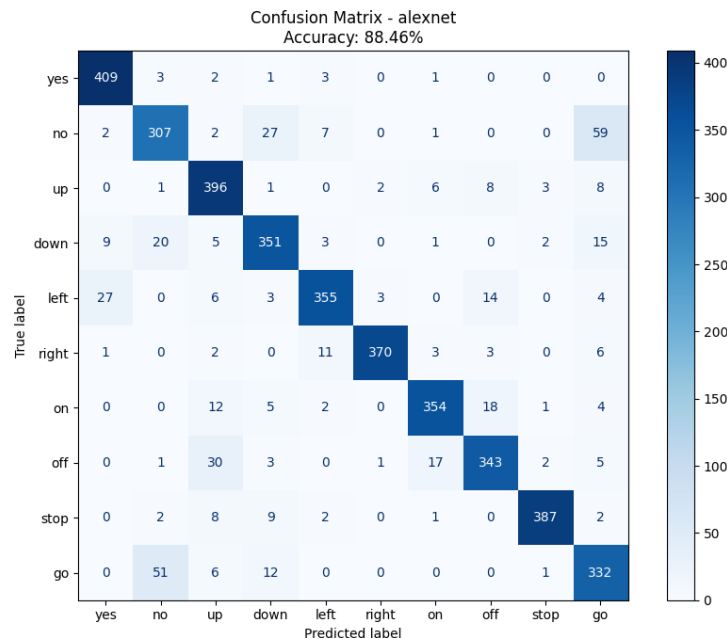


Figure 10: Matrice di confusione del modello AudioAlexNet.

La matrice di confusione di AudioAlexNet evidenzia un modello più sensibile alla complessità dell'input e meno stabile rispetto agli altri. Il risultato è coerente con il comportamento osservato nei log di training, dove AlexNet raggiunge il suo massimo prima degli altri modelli e poi non migliora più in modo significativo.

6.1 Analisi degli errori

Per capire meglio il comportamento del classificatore, tramite un codice i campioni errati vengono salvati in una catella permettono di ispezionare gli errori e di vedere la confidenza della predizione.

Questa analisi mette in evidenza un fenomeno ricorrente: molte delle errate classificazioni riguardano comandi foneticamente vicini oppure pronunciati in modo poco uniforme. Nei risultati osservati emergono errori frequenti tra **no**, **up**, **on**, **go**. Questi comandi condividono suoni simili e possono essere facilmente confusi, soprattutto se pronunciati in modo rapido o con accenti diversi. La matrice di confusione mostra che queste classi tendono a essere scambiate tra loro, segnale che il modello fatica a distinguere le sottili differenze acustiche che le caratterizzano.

6.2 Considerazioni sulla scelta del modello

Dal punto di vista pratico, LightCNN rappresenta una soluzione adatta quando si desidera un sistema semplice, veloce e facilmente addestrabile anche su hardware limitato. Au-

dioResNet è la rete con la migliore qualità finale, mentre AudioMobileNetV2 offre un compromesso interessante quando si vuole contenere il costo computazionale senza scendere troppo in accuratezza. AudioAlexNet, invece, è il modello meno convincente per questo dataset e conferma che non tutte le architetture nate per la visione si adattano allo stesso modo a una rappresentazione audio tempo-frequenza. In questo progetto il confronto tra i quattro approcci mostra bene il classico compromesso tra efficienza, robustezza e accuratezza.

7 Conclusioni

In conclusione, il progetto mostra che il riconoscimento di comandi vocali con reti neurali convoluzionali è efficace quando l'audio viene trasformato in spettrogrammi Mel. La pipeline è coerente e i risultati indicano che il sistema funziona, anche se restano margini di miglioramento.

Il confronto tra i modelli conferma che la ResNet offre la qualità migliore, mentre MobileNet rappresenta un buon compromesso tra prestazioni e costo computazionale. LightCNN resta la soluzione più semplice, mentre AlexNet si adatta meno bene a questo tipo di input.

Con un dispositivo più potente, o almeno dotato di una GPU dedicata, sarebbe stato possibile allenare il modello sul dataset completo e aggiungere anche i rumori forniti nel dataset, ottenendo un risultato più robusto.